Dynamic memory allocation

Dynamic Memory Allocation (DMA) means allocating memory during program execution (runtime)

C provides four important functions for dynamic memory allocation:

- malloc()
- calloc()
- realloc()
- free()

malloc() — Memory Allocation

malloc() allocates a block of memory of the requested size. The memory is not initialized, so it contains indeterminate values.

pointer = malloc(number_of_bytes)

```
#include <stdio.h>
#include <stdlib.h>
int main() {
    int *p;
    p = malloc(sizeof(int));
    if (p == NULL)
        {
        printf("Memory allocation failed");
        return 1;
        }
    *p = 10;
    printf("%d", *p);
    free(p);
    return 0;
}
```

10

calloc() — Contiguous Allocation

calloc() allocates memory for multiple elements and initializes all allocated bytes to zero.

pointer = calloc(number_of_elements, size_of_each_element)

```c
#include <stdio.h>
#include <stdlib.h>
int main() {
    int *p, i;
    p = calloc(3, sizeof(int));
    if (p == NULL)
        return 1;
    for (i = 0; i < 3; i++)
        printf("%d ", p[i]);
    free(p);
    return 0;
}
```

0 0 0

```c
#include <stdio.h>
#include <stdlib.h>
int main() {
    int *p, i;
    p = calloc(3, sizeof(int));
    if (p == NULL)
        return 1;
    p[0] = 10;
    p[1] = 20;
    p[2] = 30;
    for (i = 0; i < 3; i++)
        printf("%d ", p[i]);
    free(p);
    return 0;
}
```

10 20 30

realloc() — Resize Allocated Memory

realloc() changes the size of a previously allocated memory block.

pointer = realloc(pointer, new_size_in_bytes)

Increase an array from 2 to 4 integers

```c
#include <stdio.h>
#include <stdlib.h>
int main() {
    int *p, *temp, i;
    p = malloc(2 * sizeof(int));
    if (p == NULL)
        return 1;
    p[0] = 10;
    p[1] = 20;
    temp = realloc(p, 4 * sizeof(int));
    if (temp == NULL) {
        free(p);
        return 1;
    }
    p = temp;
    p[2] = 30;
    p[3] = 40;
    for (i = 0; i < 4; i++)
        printf("%d ", p[i]);
    free(p);
    return 0;
}
```

10 20 30 40

free() — Release Memory

free() releases memory that was dynamically allocated.

free(pointer)

```c
int *p = malloc(sizeof(int));

if (p != NULL) {
    *p = 50;
    printf("%d", *p);

    free(p);
    p = NULL;
}
```

50

Malloc vs Calloc Difference

Feature	malloc()	calloc()
Full Form	Memory Allocation	Contiguous Allocation
Arguments	1 argument	2 arguments
Syntax	malloc(size)	calloc(n, size)
Memory Allocation	Allocates single uninitialized block	Allocates multiple initialized blocks
Initialization	Contains garbage values	Initializes all bytes to zero (0)
Speed	Faster	Slower (due to zeroing)